

The Agent Observation Interface: Adaptive Multimodal Perception for Computer-Use Agents

Anonymous Authors
anonymous@example.com

April 16, 2026

Abstract

Current computer-use (CU) agents—closed-source and open-source alike—observe the world through periodic static screenshots and have no audio perception. This makes them unable to handle dynamic visual content (video playback, animations, transient UI events) or any audio-driven interaction (meetings, notifications, spoken instructions). We introduce the **Agent Observation Interface (AOI)**, a lightweight, model-agnostic perception layer that sits between continuous screen and audio streams and any existing image-based CU model. The AOI has three components, each scaling to zero overhead for static, silent desktop work: (1) inter-step keyframe capture that provides the CU model with additional frames from between its action steps, (2) volume-gated audio scene understanding that transcribes speech only when audio is present, and (3) CU-model-generated visual narration that converts ephemeral keyframe images into persistent text for long-term context. With zero model retraining, the AOI enables existing CU agents to handle video content, audio events, animated pages, and transient UI elements they previously could not perceive. We introduce **DynaCU-Bench**, a benchmark of 100 browser-based tasks across 10 categories that specifically require dynamic visual and/or audio perception. Five diverse CU models—Claude Sonnet 4.6, GPT-5.4, Gemini 2.5 Flash, EvoCUA-32B, and Fara-7B—equipped with the AOI achieve absolute gains of +17 to +48 percentage points over their standard screenshot-based baselines, with the strongest configuration reaching 82% task success.

1 Introduction

Computer-use (CU) agents have advanced rapidly. On OSWorld [Xie et al., 2024], scores have climbed from 12.24% (GPT-4V, April 2024) to 88.3% (Claude Sonnet 5, April 2026), surpassing the human baseline of 72.4%. A diverse ecosystem now spans closed-source systems—Claude Computer Use [Anthropic, 2024], OpenAI CUA [OpenAI, 2025], Gemini 2.5 CU [Google, 2025], Surfer 2 [H Company, 2025]—and open-source models—UI-TARS-2 [ByteDance, 2025], OpenCUA [Wang et al., 2025a], Fara-7B [Microsoft, 2025], CoAct-1 [Salesforce, 2025]. These agents can operate real desktops: filling forms, navigating websites, editing documents, and using professional software.

The blind spot. All these agents share a fundamental architectural limitation: they observe the world through *discrete static screenshots* taken every 3–5 seconds, and they are entirely deaf. Between screenshots, the agent is completely blind. This makes current CU agents incapable of understanding video content playing on screen, catching transient UI events that appear and auto-dismiss between observations, hearing meeting speech or notification sounds, or perceiving animations that convey information. A comprehensive survey of 87 CU agents [Wang et al., 2025b] does not even identify the observation modality as a gap. Quantitative evidence of this limitation

is mounting: GUI-World [Chen et al., 2025] shows video LLMs fail on all temporal GUI tasks; VideoWebArena [Jang et al., 2025] reports that long-context models perform *worse* with video than without; VideoGameBench [Zhang et al., 2025] finds even paused-game evaluation yields only 1.6% completion; and CUA-Suite [ServiceNow, 2026] demonstrates the richness of temporal dynamics that screenshot-based agents entirely miss across 55 hours of desktop recordings.

Why existing approaches fall short. Retraining CU models on video data is expensive, model-specific, and undemonstrated at scale. Using video-native models (Gemini) amounts to brute-force frame sampling that wastes tokens on redundant static frames. Native real-time multimodal models (Gemini 3.1 Flash Live [Google, 2026]) work but lock users into a single provider, offer no selective perception, and require API migration. General control frameworks like Cradle [BAAI, 2025] include pixel-level frame differencing but lack audio perception, visual narration, and the structured observation record.

Our approach. Just as SWE-agent [Yang et al., 2024] showed that the agent–computer *action* interface dramatically affects agent performance, we show that the *observation* interface is equally critical and equally underexplored. We decouple observation from action: observation runs continuously and adaptively in the background; action remains discrete. The bridge is the **Agent Observation Interface (AOI)**, a lightweight perception layer that converts continuous screen and audio streams into the sparse images and text that existing CU models already accept.

Contributions.

1. We identify the **observation interface** as a critical but overlooked bottleneck in CU agents, complementary to the action interface studied by SWE-agent.
2. We introduce the **AOI**, a model-agnostic perception layer with three components: inter-step keyframe capture, volume-gated audio scene understanding, and CU-model-generated visual narration. All three scale to zero overhead when the environment is static and silent.
3. We introduce **DynaCU-Bench**, a benchmark of 100 browser-based tasks across 10 categories that specifically require dynamic visual and/or audio perception.
4. We demonstrate that five CU models equipped with the AOI and **zero retraining** achieve absolute gains of +17 to +48 percentage points on dynamic tasks.

2 Background: The CU Agent Loop

All current CU agents follow the same observe–reason–act loop:

$$\text{repeat: } s_t \leftarrow \text{Screenshot}(), \quad a_t \leftarrow \text{CU_Model}(s_t, \tau), \quad \text{Execute}(a_t), \quad \text{Wait}(\delta) \quad (1)$$

where τ is the trajectory history and δ is a post-action buffer (typically 500 ms). The interval between observations is determined by model inference time + action execution + buffer, typically 3–5 seconds total. Everything that occurs between screenshots—visual changes, audio events, transient UI elements—is invisible to the agent.

Some models support multi-image input through screenshot history (UI-TARS: up to 5 images, Claude CU: ~ 10), but all images are post-action screenshots from prior steps. None captures what happens *between* steps.

Four categories of content are systematically missed: (i) *transient UI events*—toast notifications and dialogs that appear and vanish within one observation interval; (ii) *continuous visual media*—video playback, animated tutorials, transitioning slides; (iii) *audio events*—meeting speech, notification sounds, error alerts; (iv) *periodic visual noise*—loading spinners and blinking cursors that should be *ignored* but trigger false positives in naive frame differencing.

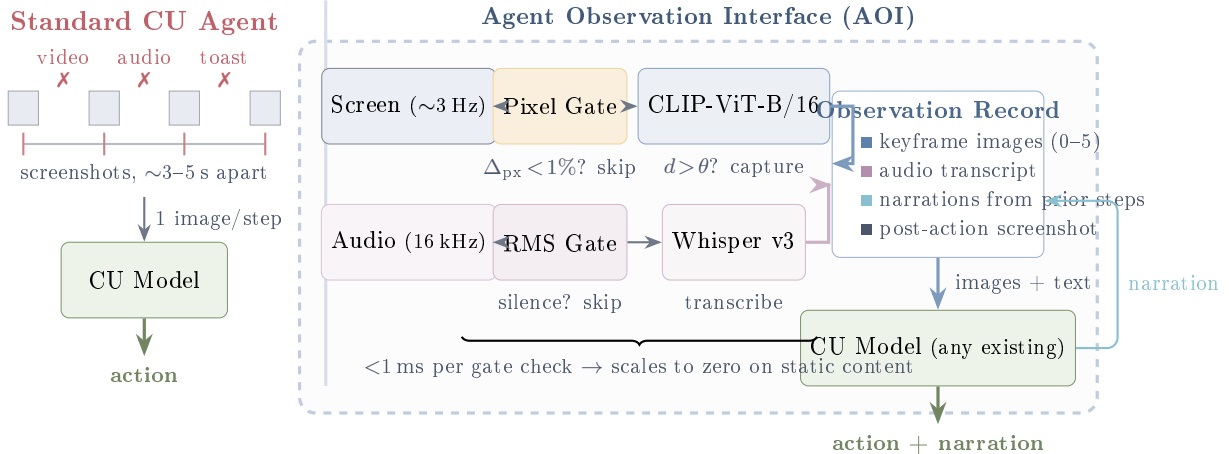


Figure 1: **Standard CU agents vs. AOI-equipped agents.** *Left:* Current agents observe through periodic screenshots ($\sim 3\text{--}5$ s apart) and are blind and deaf between observations—missing video, audio, and transient UI events (X). *Right:* The AOI continuously monitors screen and audio streams through fast gates (< 1 ms) that skip processing for static, silent content. When gates fire, keyframe extraction (shown here with optional CLIP filtering; see Section 6.5) and Whisper ASR produce images and text for the observation record. Visual narrations feed back as persistent text memory. The AOI wraps any existing CU model with zero retraining.

3 System Design

The AOI sits between the environment and any existing CU model. It observes the entire interval between agent steps and provides the CU model with additional keyframes, audio descriptions, and accumulated text context—all in the standard image + text format that every CU model already accepts. For static, silent tasks, the AOI produces nothing and behavior is identical to the standard loop. For dynamic tasks, the CU model receives strictly more information. No retraining is needed.

3.1 Architecture Overview

The AOI has three components, all governed by one principle—**scales to zero**: every component has a fast gate that short-circuits processing when there is nothing to perceive.

- (1) **Inter-step keyframe capture**: continuous screen sampling at ~ 3 Hz with gating to suppress redundant frames. Produces 0–5 keyframe images per step.
- (2) **Volume-gated audio observer**: RMS energy gate followed by ASR (Whisper large-v3). Produces a text transcription per step, only when audio is present.
- (3) **Visual narration context**: the CU model itself generates a brief text description of new visual information as a side-output of each inference call. These narrations accumulate in the trajectory and persist after keyframe images are pruned from context.

3.2 Inter-Step Keyframe Capture

Between agent steps, the screen may change (a video plays, a dialog appears, a slide transitions) or stay static. The AOI continuously samples the screen at ~ 3 Hz and selects a subset of frames to include as keyframe images in the observation record (up to 5 per step). Multiple selection strategies are possible—uniform fixed-rate sampling, pixel-level differencing, random sampling, or semantic

filtering via CLIP embeddings [Radford et al., 2021]. We evaluate all four in Section 6.5 and find that all converge to similar accuracy, so the default implementation uses simple pixel-change gating: if fewer than 1% of pixels changed since the last captured frame, the sample is skipped (cost: <1 ms on CPU). For static screens, no keyframes are produced and the component has zero overhead.

3.3 Volume-Gated Audio Observation

Current CU agents have zero audio perception. They cannot hear meeting speech, notification sounds, or error alerts. Pure ASR models (Whisper) transcribe speech but produce nothing for non-speech audio events.

Volume gate. In most computer use—file editing, form filling, silent web browsing—there is no audio. Computing RMS energy of the audio buffer is sub-millisecond. If below the silence threshold, the audio model call is skipped entirely. Cost: ~ 0 ms for the majority of steps.

When audio is present, we use Whisper large-v3 [Radford et al., 2023] for speech transcription, operating on 16 kHz mono audio captured from the system audio output via PulseAudio virtual devices. The audio buffer spans the full inter-step interval including the post-action buffer, capturing action feedback sounds (click confirmations, error beeps).

Overlapping windows. Agent step boundaries are determined by model inference latency, not by speech content. A typical 3.5-second audio chunk captures ~ 8 – 9 words at normal speaking rate, almost always falling mid-sentence. To resolve boundary artifacts, the audio buffer includes ~ 3.5 seconds of overlap from the previous interval, providing acoustic continuity across step boundaries.

3.4 Visual Narration for Long-Term Context

CU models retain only the most recent images in context. Keyframes from earlier steps are pruned. For tasks spanning many steps, the agent loses all visual memory of earlier content. Audio transcriptions persist naturally as text in the trajectory, but visual observations have no analogous persistence.

At each step, the CU model outputs both an action and a brief visual narration—a text description of what is visually new in the current keyframes. This narration is generated in the same inference call that produces the action (zero additional compute) and persists indefinitely in the trajectory, even after the corresponding images are pruned. Following the caption-then-reason principle of LLoVi [Zhang et al., 2024], narrations are generated by the CU model itself (no separate captioner) and are task-relevant rather than generic.

3.5 Observation Record

At each step, the CU model receives a structured document combining text context from recent prior steps (audio transcriptions, visual narrations, actions taken) and raw observations from the current interval (new audio transcription, any captured keyframe images, and the post-action screenshot). For static, silent tasks, this document contains only the post-action screenshot—identical to the standard loop. Figure 2 shows a concrete example.

4 Implementation

The AOI is implemented in Python ($\sim 2,600$ lines) as a modular layer that wraps any CU model. Screen capture uses a headless browser (Playwright) at ~ 3 Hz. Audio capture and injection use PulseAudio virtual devices (virtual speaker and monitor source) at 16 kHz mono. CLIP-ViT-B/16

Observation Record — Step N (Meeting Task B-E1)

```
# CONTEXT -- text from prior steps (no images)
Step N-1 ( $t \approx 18.5\text{-}22.0\text{ s}$ ):
AUDIO: "Here's the team. As you can see we
have strong cross-functional coverage."
VISUAL: "Meeting slide shows Project Team
with 6 members listed in a grid."
ACTION: wait()

# NEW -- current interval observations
Step N ( $t \approx 22.0\text{-}25.5\text{ s}$ ):
AUDIO: "And I want to mention, we've moved
the launch date to April 28th.
Please update your calendars."
[IMAGE: post-action screenshot]

# TASK
Read the meeting slides and listen to the
discussion. What is the new launch date?
Enter it in the Launch Date field.
```

Figure 2: Example observation record sent to the CU model at step N of a meeting task. The **context** section provides text from prior steps (audio transcriptions and visual narrations persist after images are pruned). The **new** section contains the current audio transcription and the post-action screenshot (image). In this case, no keyframes were captured (the slide did not change), so only the screenshot is included as an image. The task instruction is appended at the end.

runs on GPU ($\sim 150\text{ MB}$ VRAM). Whisper large-v3 runs as a separate HTTP service on CPU to avoid GPU contention with the CU model. Audio synthesis for benchmark tasks uses the Web Speech API (`speechSynthesis`).

CU models are accessed through their native APIs. Cloud models (Claude Sonnet 4.6, GPT-5.4, Gemini 2.5 Flash) use HTTP APIs. Local models (EvoCUA-32B, Fara-7B) are served via vLLM with 4-bit quantization on a single NVIDIA RTX PRO 6000 (96 GB VRAM).

5 DynaCU-Bench

5.1 Design Principles

Existing CU benchmarks (OSWorld, WebArena) consist entirely of tasks solvable from static screenshots. We introduce **DynaCU-Bench**, a benchmark of 100 browser-based tasks that specifically require dynamic visual and/or audio perception—tasks that are unsolvable from static screenshots alone.

Design principles: (i) tasks must *require* temporal visual or audio perception; (ii) tasks represent realistic computer-use scenarios, not artificial constructs; (iii) tasks execute in a reproducible headless browser environment; (iv) tasks cover all four dynamic content categories from Section 2; (v) tasks include difficulty stratification (3 easy, 4 medium, 3 hard per category).

5.2 Task Categories

DynaCU-Bench spans 10 categories across three capability axes: audio perception (A), visual-temporal perception (V), and real-time interaction (I). Each category has 10 tasks.

Table 1: DynaCU-Bench task categories. A = audio perception, V = visual-temporal perception, I = real-time interaction. Each category contains 3 easy, 4 medium, and 3 hard tasks.

Cat.	Domain	Axes	Description
A	Podcast comprehension	A	Listen to audio narration, extract facts or answer questions
B	Meeting participation	A+V+I	Follow meeting with slides and speech, take notes or act
C	Video/screencast	V	Watch terminal recordings or IDE demos with auto-advancing frames
D	Carousel/animation	V	Perceive CSS transitions, rotating content, product carousels
E	Live dashboard	V	Monitor real-time updating dashboards with <code>setInterval</code> changes
F	Transient UI events	V	Respond to toast notifications, auto-dismissing banners
G	Phone/voice call	A+I	Listen to <code>speechSynthesis</code> calls and respond
H	Interview/voice	A+I	Answer questions via <code>SpeechRecognition</code> , complete interviews
I	Collaborative editing	V+I	Handle remote changes in collaborative document editors
J	Browser games	V+I	Play interactive games requiring temporal attention

5.3 Evaluation Protocol

Each task executes in a headless Chromium browser (Playwright) with PulseAudio virtual devices for audio I/O. Audio content is delivered via the Web Speech API (`speechSynthesis`) with edge-TTS for high-quality voices. Agent audio input is injected into a virtual microphone via `pacat`.

Three evaluation modes are used: *DOM-based* (93 tasks): a deterministic JavaScript function `getTaskResult()` returns the expected value; *LLM-judged* (1 task): a judge model scores the agent’s response against a rubric; *Hybrid* (6 tasks): DOM gate (did the agent act?) combined with LLM quality assessment. All tasks allow a maximum of 15 agent steps.

6 Evaluation

6.1 Setup

CU models. We evaluate five models spanning the capability and scale spectrum: (i) Claude Sonnet 4.6 [Anthropic, 2024] (closed-source, OSWorld 72.5%); (ii) GPT-5.4 [OpenAI, 2025] (closed-source, OSWorld 75.0%); (iii) Gemini 2.5 Flash [Google, 2025] (closed-source); (iv) EvoCUA-32B (open-source, 32B); (v) Fara-7B [Microsoft, 2025] (open-source, 7B).

Observation configurations. We compare 8 observation modes:

- (1) **Standard**: one screenshot per step, no audio (current paradigm baseline).
- (2) **Uniform 1 FPS**: capture at fixed 1 FPS, feed recent N frames.
- (3) **Uniform 3 FPS**: capture at fixed 3 FPS, feed recent N frames.
- (4) **Pixel-diff**: capture when pixel change ratio exceeds threshold, no CLIP filtering.
- (5) **Random keyframes**: randomly sample the same number of frames as our method.
- (6) **AOI (visual only)**: adaptive keyframes (CLIP-based), no audio.

Table 2: DynaCU-Bench results: task success rate (%) across all observation configurations. **Bold** indicates the best result per model. Δ shows absolute gain over standard baseline. All ablation modes tested with Claude Sonnet 4.6. “Claude 4.6” abbreviates Claude Sonnet 4.6 throughout tables and figures.

Configuration	Claude 4.6	GPT-5.4	Gemini 2.5	EvoCUA-32B	Fara-7B
Standard (screenshot)	38	37	21	18	17
Uniform 1 FPS	58	—	—	—	—
Uniform 3 FPS	56	—	—	—	—
Pixel-diff only	58	—	—	—	—
Random keyframes	56	—	—	—	—
AOI (visual only)	58	—	—	—	—
AOI (visual + ASR)	64	—	—	—	—
AOI (full)	82	57	69	55	34
Δ (AOI full — standard)	+44	+20	+48	+37	+17

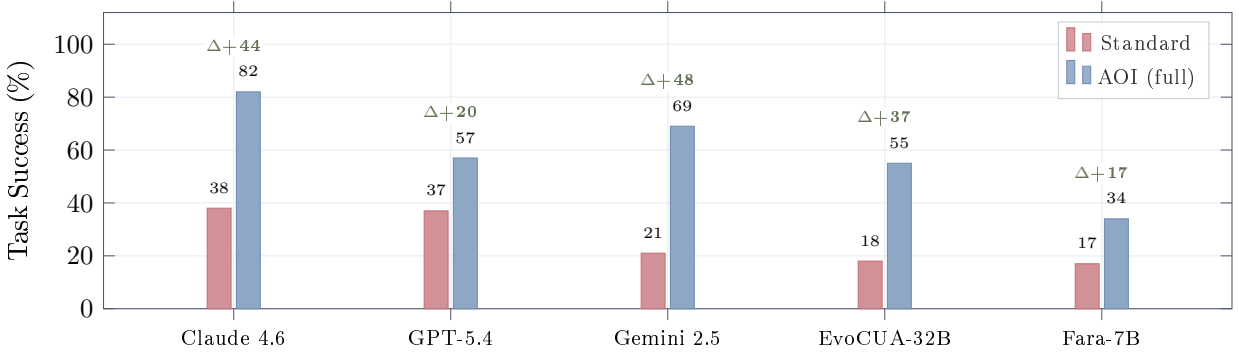


Figure 3: **Main results on DynaCU-Bench (100 tasks).** The AOI produces large gains across all five CU models without any retraining. Green numbers show absolute improvement in percentage points. Gemini 2.5 Flash shows the largest absolute gain (+48 pp) and relative gain (3.3 $\times$), while Claude achieves the highest absolute score (82%).

(7) **AOI (visual + ASR)**: adaptive keyframes + Whisper speech transcription.

(8) **AOI (full)**: adaptive keyframes + Whisper ASR + visual narration.

Ablation configurations (2)–(7) are evaluated with Claude Sonnet 4.6; the full AOI and standard baseline are evaluated with all five models.

6.2 Main Results

Table 2 and Figure 3 present the main results. The AOI produces large, consistent gains across all five models. *Note on statistical power*: Each configuration is evaluated once on 100 binary-outcome tasks (no repeated trials), so small differences (e.g., 56% vs. 58%, a 2-task gap) are within sampling noise. We focus interpretation on large, consistent effects across models rather than fine-grained rankings between ablation modes.

Claude Sonnet 4.6 improves from 38% to 82% (+44 pp) with the full AOI. This is the strongest absolute result. The ablation modes reveal that all inter-step visual capture methods converge to 56–58% (a +18–20 pp gain from additional frames alone, regardless of selection method), adding

Table 3: Per-category task success (pass/10) for standard vs. AOI full across all five models. Categories are grouped by primary perception axis: A = audio, V = visual-temporal, I = real-time interaction.

Cat.	Axes	Claude 4.6		GPT-5.4		Gemini 2.5		EvoCUA-32B		Fara-7B	
		Std	AOI	Std	AOI	Std	AOI	Std	AOI	Std	AOI
A (Podcast)	A	0	9	0	3	0	8	0	8	0	3
B (Meeting)	A+V+I	2	10	2	3	2	8	1	6	1	5
C (Video)	V	4	9	5	8	5	6	3	6	3	5
D (Carousel)	V	4	9	6	7	4	7	3	4	1	3
E (Dashboard)	V	8	8	6	9	1	9	2	4	2	2
F (Transient)	V	4	9	2	8	0	9	0	5	4	3
G (Phone)	A+I	1	9	1	4	0	8	0	8	1	4
H (Interview)	A+I	7	9	6	7	5	7	6	8	1	3
I (Collab)	V+I	3	8	4	5	2	6	2	5	2	4
J (Games)	V+I	5	2	5	3	2	1	1	1	2	2
Total		38	82	37	57	21	69	18	55	17	34

ASR reaches 64%, and visual narration provides the largest single-component gain, reaching 82%. The +18 pp gap between visual+ASR and full AOI demonstrates that *retaining* visual information as text is as important as *capturing* it.

GPT-5.4 improves from 37% to 57% (+20 pp). The gain is consistent with Claude’s but the absolute ceiling is lower, suggesting GPT-5.4’s CU capabilities are the binding constraint once perception is adequate.

Gemini 2.5 Flash scores 21% on the standard baseline despite being a video-native model with built-in streaming capabilities. This confirms that video-native architectures do not automatically solve the observation problem—the standard CU agent loop still constrains Gemini to periodic screenshots. With the AOI, Gemini improves to 69% (+48 pp)—the largest absolute gain of any model tested, demonstrating that a video-native model benefits even more from structured observation than text-centric models.

EvoCUA-32B improves from 18% to 55% (3× improvement, +37 pp). This open-source 32B model, which barely functions on dynamic tasks without the AOI, becomes competitive with GPT-5.4’s AOI-equipped performance.

Fara-7B improves from 17% to 34% (+17 pp), doubling its score. As the smallest model (7B parameters), it is most constrained by reasoning capacity, but still benefits substantially from improved perception.

6.3 Per-Category Analysis

Table 3 and Figure 5 reveal category-level patterns:

Audio categories (A, B, G) show the largest gains. Standard baselines score 0–2/10 on podcast, meeting, and phone tasks across all models—these tasks are impossible without audio perception. The AOI unlocks 3–10/10 depending on the model, with Claude achieving near-perfect scores (9/10 or 10/10).

Visual-temporal categories (C, D, F) benefit consistently. Video, carousel, and transient UI tasks improve by 2–7 points across models. Category E (dashboard) shows smaller gains because dashboard tasks often have sufficient information in single screenshots—the dynamic updates are periodic and the current value is always visible.

Task B-E1: Meeting — “What is the new launch date?”	
Standard (screenshot-only) — FAIL 15 steps, 51.2 s 1. <code>wait()</code> — sees static slide 2. <code>wait()</code> — same slide 3. <code>wait()</code> — same slide 4. <code>wait()</code> — same slide 5. <code>wait()</code> — same slide ⋮ 15. <code>fill(#launchDate, “TBD”)</code> × guesses Agent never hears the spoken date. Waits 15 steps, then guesses.	AOI (full) — PASS 2 steps, 26.2 s 1. <code>wait()</code> Audio: “Here’s the team. As you can see we have strong cross-functional coverage.” Narration: “Meeting slide shows Project Team with 6 members.” 2. <code>fill(#launchDate, “April 28th”)</code> ✓ Audio: “We’ve moved the launch date to April 28th. Please update.” Narration: “Audio announced the new launch date.” Agent hears “April 28th” and fills the form immediately.
Task C-E1: Video — “What was the first command in the terminal recording?”	
Standard — PASS (6 steps) 1. <code>wait()</code> — recording playing 2. <code>wait()</code> — missed first command 3. <code>wait()</code> — scrolling through 4. <code>wait()</code> 5. <code>wait()</code> 6. <code>fill(...)</code> ✓ lucky timing	AOI — PASS (1 step) 1. <code>fill(#answerInput, “git clone ...”)</code> ✓ Narration: “The terminal recording shows the first command was git clone...” Narration captures the command from the recording immediately.

Figure 4: Trajectory comparison on two representative tasks. **Top:** Meeting task (B-E1) where the launch date is spoken aloud. The standard agent cannot hear it and guesses after 15 futile `wait()` steps. The AOI agent transcribes the audio, hears “April 28th,” and acts in 2 steps. **Bottom:** Video task (C-E1) where the answer appears in a terminal recording. The standard agent needs 6 steps with lucky timing; the AOI agent’s narration captures the content in 1 step.

Category J (games) is an exception. Browser games require not just perception but fast, precise motor actions and strategic planning. The AOI provides better perception but does not accelerate the agent’s action latency, which is the binding constraint for real-time games. Claude’s score drops from 5 to 2 with the AOI. Category J has the highest step count (118 steps, 11.8 per task) and high keyframe density (0.55 KF/step; Table 6), resulting in substantial observation overhead: CLIP encoding and keyframe processing add $\sim 2\text{--}3\text{s}$ per step on top of the model’s $\sim 3\text{s}$ inference latency, roughly doubling the agent’s effective reaction time. For time-sensitive game tasks where actions must occur within specific windows, this added latency causes the agent to miss deadlines it would otherwise meet. This confirms that the AOI is designed for human-paced computer use, not real-time interactive applications.

EvoCUA-32B shows disproportionate gains on audio tasks. The model jumps from 0/10 to 8/10 on both podcast (A) and phone (G) categories, suggesting that audio perception particularly benefits models whose visual grounding is weaker.

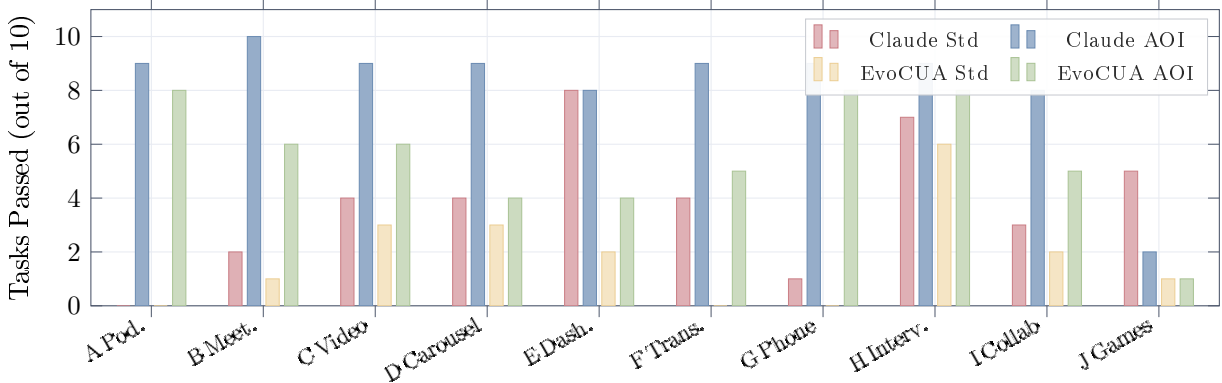


Figure 5: **Per-category results for Claude Sonnet 4.6 and EvoCUA-32B.** Audio-dependent categories (A, B, G) show the largest gains—from near-zero to 6–10/10. Visual-temporal categories (C, D, F) improve consistently. Category J (games) is the exception: real-time action latency, not perception, is the bottleneck. Category E (dashboard) shows minimal change for Claude because current values are visible in single screenshots.

Table 4: Task success by difficulty level (standard → AOI full). Easy: 30 tasks (3 per category). Medium: 40 tasks (4 per category). Hard: 30 tasks (3 per category).

Model	Standard			AOI Full		
	Easy	Med	Hard	Easy	Med	Hard
Claude 4.6	17/30	15/40	6/30	28/30	34/40	20/30
GPT-5.4	15/30	17/40	5/30	21/30	26/40	10/30
Gemini 2.5	10/30	9/40	2/30	24/30	33/40	12/30
EvoCUA-32B	6/30	8/40	4/30	20/30	26/40	9/30
Fara-7B	9/30	5/40	3/30	19/30	11/40	4/30

6.4 Difficulty Breakdown

Table 4 shows that the AOI lifts all difficulty levels. For Claude, easy task success rises from 57% to 93%, medium from 38% to 85%, and hard from 20% to 67%. The pattern is consistent across models: the AOI unlocks perception-gated tasks at every difficulty, but hard tasks remain challenging because they require both perception *and* sophisticated reasoning. Fara-7B’s small gain on medium/hard (5→11, 3→4) compared to its large gain on easy (9→19) suggests that reasoning capacity, not perception, becomes the bottleneck for the 7B model on harder tasks.

6.5 Ablation: Observation Mode Analysis

Table 5 and Figure 6 reveal the contribution of each AOI component:

Selection method does not affect accuracy (56–58%). We evaluated five visual-only keyframe strategies: (a) *Uniform 1FPS*: capture at a fixed rate; (b) *Uniform 3FPS*: higher fixed rate; (c) *Pixel-diff*: capture when >1% of pixels change; (d) *Random*: randomly sample the same number of frames as adaptive methods; (e) *CLIP-based adaptive*: two-stage filter (pixel gate → CLIP-ViT-B/16 cosine distance exceeding threshold θ ; Algorithm 1). All five converge to a narrow performance band of 56–58%. This is a key finding: the dominant factor for visual perception is capturing *any* inter-step frames, not *how* they are selected. The +18–20 pp gain over the single-

Table 5: Ablation study on Claude Sonnet 4.6: contribution of each AOI component. All configurations include per-category breakdown for categories most affected.

Mode	Total /100	A	B	C	G	H
(pass/10, selected categories)						
Standard	38	0	2	4	1	7
Uniform 1 FPS	58	0	4	10	1	8
Uniform 3 FPS	56	2	5	8	1	7
Pixel-diff	58	1	5	9	1	8
Random keyframes	56	1	6	8	1	8
AOI visual only	58	1	5	9	1	8
AOI visual + ASR	64	1	4	10	4	9
AOI full	82	9	10	9	9	9

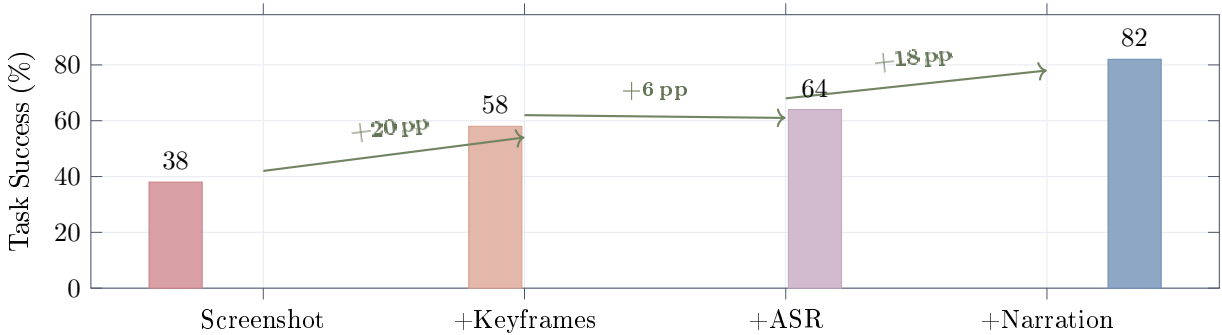


Figure 6: **Ablation: progressive contribution of each AOI component** (Claude Sonnet 4.6). Inter-step keyframes provide +20 pp (any selection method achieves this; see Table 5). ASR adds +6 pp (hearing speech and sounds). Visual narration adds +18 pp—the largest single-component gain—demonstrating that *retaining* visual information as persistent text is as important as *capturing* it.

screenshot baseline comes from breaking the agent’s blindness between steps, regardless of whether frames are chosen uniformly, randomly, or adaptively. This result simplifies the practical recommendation: practitioners can use whatever keyframe strategy fits their latency and compute budget.

Adding ASR (+6 pp). Whisper speech transcription lifts performance from 58% to 64%. The gain concentrates on audio-dependent categories (G: 1→4, H: 8→9). Categories A and B show minimal improvement because podcast and meeting comprehension require understanding the *content* of speech over multiple steps, not just detecting that speech occurred.

Full AOI (+18 pp over visual+ASR). The full AOI with visual narration reaches 82%. The large gap from 64% to 82% demonstrates that visual narration—converting ephemeral keyframe images into persistent text—is critical for tasks requiring recall of earlier observations. The improvement is especially pronounced on podcast (A: 1→9) and meeting (B: 4→10) tasks, where narration enables the agent to accumulate information across many steps.

Caveat: This gain may conflate two effects: (a) the persistent text memory that survives keyframe pruning, and (b) a chain-of-thought effect from the narration generation itself, which forces the model to explicitly reason about visual content before selecting an action. Disentangling these effects would require an ablation where the model generates narrations but they are discarded rather than retained—we leave this to future work.

Algorithm 1 CLIP-Based Adaptive Keyframe Extraction

Require: Screen sample x_t , anchor embedding e_{anchor} , pixel threshold α , CLIP threshold θ

```
1:  $\Delta_{\text{px}} \leftarrow \text{PixelChangeRatio}(x_t, x_{\text{prev}})$ 
2: if  $\Delta_{\text{px}} < \alpha$  then
3:   return SKIP {Stage 1: no pixel change}
4: end if
5:  $e_t \leftarrow \text{CLIP}(x_t)$ 
6:  $d \leftarrow 1 - \cos(e_t, e_{\text{anchor}})$ 
7: if  $d > \theta$  then
8:    $e_{\text{anchor}} \leftarrow e_t$  {Re-anchor}
9:   return CAPTUREKEYFRAME( $x_t, t$ )
10: end if
11: return SKIP {Stage 2: below semantic threshold}
```

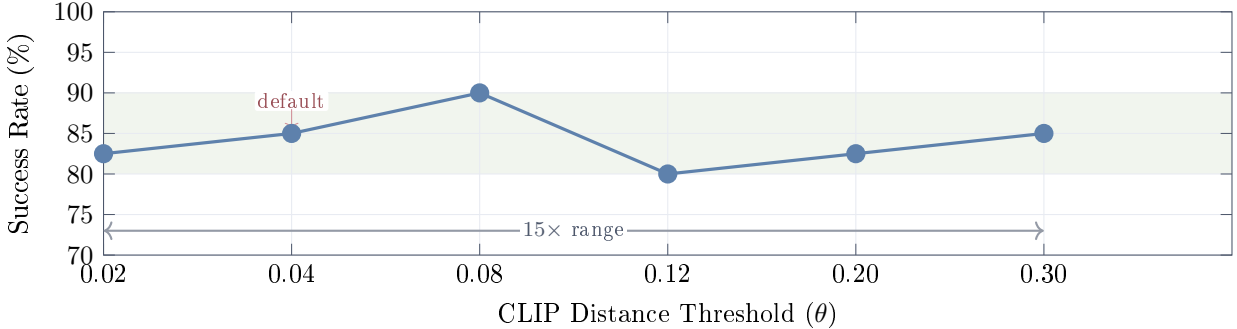


Figure 7: **CLIP threshold (θ) sensitivity** on 40 visual tasks (categories C–F) with Claude Sonnet 4.6. Performance remains within an 80–90% band (shaded) across a 15 $\times$ range of θ , from 0.02 (capturing minor changes) to 0.30 (only major visual shifts). The bimodal distribution of CLIP distances—near-zero for unchanged content, large for meaningful changes—makes the method naturally insensitive to θ .

7 Analysis

7.1 CLIP Threshold Sensitivity

Figure 7 shows the CLIP threshold θ sweep across a 15 $\times$ range from 0.02 to 0.30, evaluated on the 40 visual tasks (categories C–F) where keyframe selection is relevant. Success rate remains within an 80–90% band throughout, demonstrating that for practitioners who choose the CLIP-based variant, threshold selection is not critical. There is no cliff: even at $\theta = 0.30$ (very aggressive filtering that captures only major visual changes), performance remains at 85%. At $\theta = 0.02$ (very sensitive, capturing small changes), performance is 82.5%—slightly lower, likely because additional low-relevance keyframes dilute the model’s attention. The slight peak at $\theta = 0.08$ (90%) suggests a mild sweet spot, but the practical implication is that any reasonable value of θ will work.

7.2 Per-Step Observation Statistics

Table 6 and Figure 8 show the per-step observation statistics for the AOI. The data confirms the “scales to zero” design: audio-only categories (G, H) produce zero keyframes, while visual-only

Table 6: Per-step observation statistics for Claude Sonnet 4.6 + AOI full, broken down by task category. %KF = percentage of steps where at least one keyframe was captured. KF/Step = average keyframes per step. %Audio = percentage of steps with audio transcription. Audio-heavy categories (A, B, G, H) show 27–74% audio activation. Visually dynamic categories (D, J) average 0.55–1.38 keyframes per step. Static categories (G, H) produce zero keyframes—the pixel and CLIP gates correctly suppress extraction when only audio changes.

Cat.	Domain	Steps	%KF	Tot. KF	KF/Step	% Audio
A	Podcast	35	8.6	3	0.09	42.9
B	Meeting	34	50.0	18	0.53	73.5
C	Video	39	28.2	12	0.31	2.6
D	Carousel	61	60.7	84	1.38	0.0
E	Dashboard	52	28.8	18	0.35	0.0
F	Transient	38	15.8	7	0.18	0.0
G	Phone	38	0.0	0	0.00	26.3
H	Interview	27	0.0	0	0.00	37.0
I	Collab	39	10.3	4	0.10	17.9
J	Games	118	36.4	65	0.55	0.0
All		481	28.3	211	0.44	14.1

categories (D, E, F, J) produce zero audio activations. Category D (carousel) has the highest keyframe density at 1.38 per step because carousel animations produce frequent visual changes. Category A (podcast) has the lowest keyframe rate (0.09) because podcast tasks involve a mostly static UI with audio content. Overall, only 28.3% of steps produce keyframes and 14.1% activate audio processing—the gates are effective at suppressing unnecessary computation.

7.3 Efficiency

Table 7 reveals an important pattern: the AOI reduces the number of steps needed to complete tasks despite adding per-step observation overhead. Claude uses 4.8 steps on average with AOI vs. 10.7 without—fewer than half as many steps. This occurs because better perception leads to more informed actions, reducing trial-and-error exploration.

The observation overhead (12–32 s per task) is offset by reduced model inference costs from fewer steps. For Claude, total task time increases modestly (40.6 s $\rightarrow$ 46.2 s) despite the per-step overhead, because the step count drops by 55%. For Gemini 2.5 Flash, step count drops from 11.5 to 5.4 (53% reduction), with total time increasing modestly from 55.3 s to 61.9 s. For EvoCUA-32B, total time actually *decreases* (117.0 s $\rightarrow$ 113.9 s) because the model’s high inference latency ($\sim$ 10 s/step) makes step reduction dominant.

7.4 Static Task Verification

A key design claim is that the AOI “scales to zero” on static, silent desktop work—adding no unnecessary overhead when the screen is static and no audio is present. We verify this with 10 purely static HTML tasks (reading text, clicking buttons, filling forms, navigating links) containing no animations, timers, video, or audio.

Table 8 confirms the design property: across all 17 AOI steps on static content, the pixel gate detected no meaningful screen changes (total keyframes = 0) and the volume gate detected no audio (audio segments = 0). The observation pipeline runs but produces empty observations,

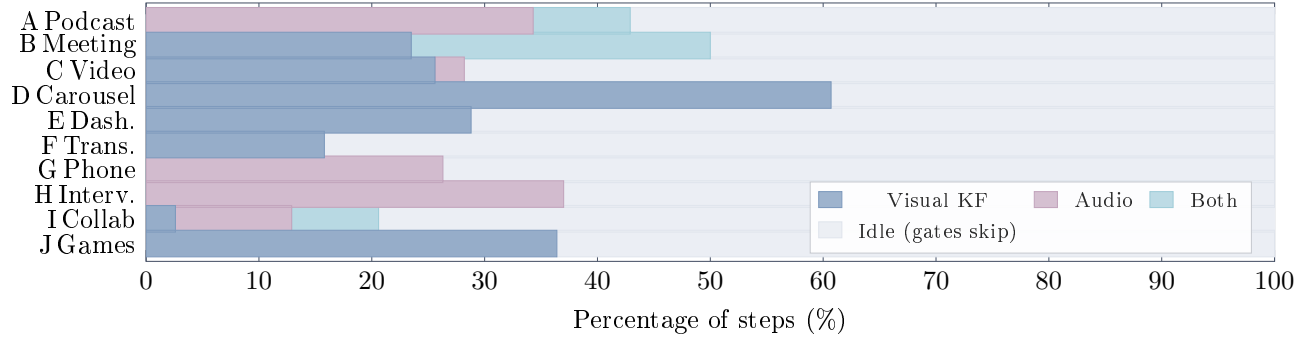


Figure 8: **Observation activity per category.** Stacked bars show the percentage of steps where each observation channel fires. The gray “idle” portion represents steps where both gates suppress processing—the AOI does nothing beyond sub-millisecond gate checks. Categories naturally cluster: D (carousel) is highly visual, G/H (phone/interview) are audio-only, B (meeting) uses both channels. On average, 57.6% of steps are idle—confirming the “scales to zero” design.

adding only the fixed sampling cost (~ 2 s per step for screenshot capture and CLIP embedding) with no keyframe images inserted into the context. The AOI configuration scores 9/10 vs. 7/10 for standard mode and uses fewer steps (1.7 vs. 3.5 average). Since no additional observations are injected, this improvement likely stems from the AOI’s structured observation record prompt format rather than from the perception layer itself—a confound we note but do not attempt to disentangle, as the primary purpose of this experiment is verifying the “scales to zero” gate behavior.

7.5 Failure Modes

We identify three systematic failure modes:

Fast visual transitions. Events shorter than ~ 300 ms (one sample interval at 3 Hz) may fall entirely between samples and be missed. This primarily affects game tasks (category J) where targets appear briefly.

Context window limits. On keyframe-heavy tasks (categories D, E with many visual changes), accumulated keyframe images can exceed the CU model’s context window. We observed context overflow errors at 8,192 tokens for local models on some hard tasks. This is a model-side constraint, not an AOI limitation.

Reasoning bottleneck. For smaller models (Fara-7B), improved perception does not help with tasks requiring multi-step reasoning, arithmetic, or complex instruction following. The AOI provides better *inputs* but cannot improve the model’s reasoning over those inputs.

8 Related Work

Computer-use agents. The CU agent landscape spans closed-source systems [Anthropic, 2024, OpenAI, 2025, Google, 2025, H Company, 2025] and open-source models [ByteDance, 2025, Wang et al., 2025a, Microsoft, 2025, Alibaba, 2025, Salesforce, 2025]. Agent S3 [Simular AI, 2025] was the first open-source agent to surpass the human baseline on OSWorld. A comprehensive survey [Wang et al., 2025b] classifies 87 agents and 33 datasets, identifying six gaps—but does not identify the observation modality limitation. All of these agents operate through the screenshot–reason–act loop. None addresses dynamic visual content or audio perception.

Table 7: Efficiency statistics across observation configurations. Avg Steps = mean steps per task (lower is better—fewer steps means faster task completion). Obs Overhead = total observation processing time per task, including screenshot capture (~ 2 s/step baseline for both standard and AOI) plus CLIP encoding, keyframe selection, and audio processing for AOI configurations.

Configuration	Avg Steps	Avg Time (s)	Model Lat. (s)	Obs Over. (s)
<i>Claude Sonnet 4.6</i>				
Standard	10.7	40.6	26.3	2.5
AOI full	4.8	46.2	15.2	12.0
<i>GPT-5.4</i>				
Standard	10.0	26.6	14.4	2.4
AOI full	7.6	50.6	11.2	17.8
<i>Gemini 2.5 Flash</i>				
Standard	11.5	55.3	40.4	2.6
AOI full	5.4	61.9	20.6	20.6
<i>EvoCUA-32B</i>				
Standard	9.3	117.0	99.0	2.3
AOI full	5.3	113.9	72.0	22.6
<i>Fara-7B</i>				
Standard	13.4	35.1	24.0	2.5
AOI full	9.3	90.7	17.0	31.6

Table 8: Static task verification. On 10 purely static tasks, the AOI extracts zero keyframes and zero audio segments, confirming that the pixel gate and volume gate correctly suppress observation when no dynamic content is present. The AOI does not degrade performance on static work.

Mode	Pass/10	Avg Steps	Total KF	Audio Seg.
Standard	7	3.5	0	0
AOI full	9	1.7	0	0

Real-time multimodal models. Gemini 3.1 Flash Live [Google, 2026] processes continuous audio/video/text streams in real time. This monolithic approach embeds continuous perception into a specific model. Our work is complementary: model-agnostic adaptive perception as an external layer. Flash Live bundles perception with reasoning in a single model; the AOI decouples them, allowing any CU model to benefit.

Video understanding for GUI. GUI-World [Chen et al., 2025], VideoWebArena [Jang et al., 2025], CUA-Suite [ServiceNow, 2026], and Gaia2 [Gaia Team, 2026] identify the observation problem through benchmarking. MemGUI-Bench [MemGUI Team, 2026] reveals memory gaps in GUI agents. These benchmarks identify the problem but do not propose solutions compatible with existing CU agents.

Keyframe selection. VideoTree [Wang et al., 2025c] and AKS [Tang et al., 2025] use CLIP-based frame selection for offline video QA. Apollo [Apollo Team, 2025] finds that FPS sampling outperforms uniform sampling and identifies optimal token budgets. All prior work targets offline video understanding; our contribution is real-time observation filtering for CU agents, where the two-stage design is critical for <10 ms latency.

Adaptive middleware for GUI agents. Cradle [BAAI, 2025] includes frame difference analysis for games—the closest predecessor to the AOI’s perception layer—but uses pixel-level differencing

without semantic filtering and has no audio processing. ScreenLLM [ScreenLLM Team, 2025] captures stateful screen schemas with keyframe extraction, complementing visual narration but without audio or adaptive observation. StreamAgent [StreamAgent Team, 2026] and Dispider [Dispider Team, 2025] separate perception from response generation for streaming video, paralleling the AOI’s architecture, but neither addresses CU agent observation.

Structured agent interfaces. WebMCP [Google and Microsoft, 2026] proposes structured AI–website interactions via declarative APIs. Where WebMCP requires website cooperation, the AOI enables perception on arbitrary, unmodified web content.

9 Limitations

Action latency is unsolved. The AOI extends perception but does not accelerate decision-making. CU models still require 1–5 seconds per inference step, making the approach unsuitable for real-time games or fast interactive applications (as evidenced by category J results).

Temporal resolution floor. Sampling at ~ 3 Hz means events shorter than ~ 300 ms may be missed. For human-paced computer use (meetings, web browsing, presentations), this resolution is sufficient. For high-speed visual content, information is irretrievably lost.

Narration quality is bounded by the CU model. Long-term visual memory depends on the CU model producing accurate text descriptions. Errors—a misread number, a hallucinated UI element—are permanently encoded in the trajectory.

Audio limited to speech transcription. Our implementation uses Whisper for speech-only ASR rather than a multimodal audio model. Non-speech audio events (notification sounds, alert beeps) are not transcribed. A multimodal audio model (e.g., Qwen3 Omni) could extend coverage to the full audio scene.

Benchmark scope. DynaCU-Bench tasks execute in a controlled browser environment with synthesized audio. Performance on real-world dynamic content (live video calls, real YouTube, native desktop applications) may differ.

10 Conclusion

Computer-use agents cannot perceive dynamic content because their observation is limited to periodic static screenshots and they have no audio perception. We show this is an *observation problem*, *not a model problem*, and introduce the Agent Observation Interface—a model-agnostic perception layer that is transparent for static tasks, adaptive for dynamic content, and compatible with any existing CU model without retraining. On DynaCU-Bench, five diverse CU agents equipped with the AOI achieve absolute gains of +17 to +48 percentage points, with the strongest configuration reaching 82% task success. A key finding is that keyframe selection method does not significantly affect accuracy: all strategies converge to $\sim 58\%$, indicating that the critical factor is capturing *any* inter-step frames, not how they are selected. The largest single contributor to the full system’s gains is visual narration (+18 pp), demonstrating that *retaining* visual information as persistent text is as important as *capturing* it. The bottleneck in computer-use agents has been reasoning and action; we show that observation is equally critical and introduce a practical solution.

References

Alibaba. GUI-Owl / Mobile-Agent-v3: Fundamental agents for GUI automation. *arXiv preprint arXiv:2508.15144*, 2025.

- Anthropic. Claude computer use. <https://docs.anthropic.com/en/docs/agents-and-tools/computer-use>, 2024.
- Apollo Team. Apollo: An exploration of video understanding in large multimodal models. In *CVPR*, 2025.
- BAAI. Cradle: Empowering foundation agents towards general computer control. In *ICML*, 2025.
- ByteDance. UI-TARS-2: Pioneering automated GUI interaction with native agents, 2025.
- Dongping Chen et al. GUI-World: A dataset for GUI-oriented multimodal LLM-based agents. In *ICLR*, 2025.
- Dispider Team. Dispider: Enabling video LLMs with active real-time interaction via disentangled perception, decision, and reaction. In *CVPR*, 2025.
- Gaia Team. Gaia2: Dynamic, asynchronous environments for LMM evaluation. In *ICLR*, 2026.
- Google. Gemini 2.5 computer use, 2025.
- Google. Gemini 3.1 flash live. <https://ai.google.dev/gemini-api/docs/live>, 2026.
- Google and Microsoft. WebMCP: Proposed W3C standard for structured AI agent interactions with websites, 2026.
- H Company. Surfer 2: The next generation of cross-platform computer use agents. *arXiv preprint arXiv:2510.19949*, 2025.
- Lawrence Jang et al. VideoWebArena: Evaluating long context multimodal agents with video tasks. In *ICLR*, 2025.
- MemGUI Team. MemGUI-Bench: Memory-centric benchmark for mobile GUI agents. *arXiv preprint arXiv:2602.06075*, 2026.
- Microsoft. Fara-7b: An efficient agentic model for computer use. *arXiv preprint arXiv:2511.19663*, 2025.
- OpenAI. Computer-using agent (CUA). <https://openai.com/index/computer-using-agent/>, 2025.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *ICML*, 2023.
- Salesforce. CoAct-1: Computer-using multi-agent system with coding actions. *arXiv preprint arXiv:2508.03923*, 2025.
- ScreenLLM Team. ScreenLLM: Stateful screen schema for efficient action understanding and prediction. In *WWW*, 2025.
- ServiceNow. CUA-Suite / VideoCUA. *arXiv preprint arXiv:2603.24440*, 2026.

- Simular AI. Agent S3. <https://www.simular.ai/agent-s3>, 2025.
- StreamAgent Team. StreamAgent: Towards anticipatory agents for streaming video understanding. In *ICLR*, 2026.
- Jianxiong Tang et al. Adaptive keyframe sampling for long video understanding. In *CVPR*, 2025.
- Boyuan Wang et al. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025a. NeurIPS 2025 Spotlight.
- Xinyi Wang et al. A comprehensive survey of agents for computer use. *arXiv preprint arXiv:2501.16150*, 2025b.
- Ziyang Wang et al. VideoTree: Adaptive tree-based video representation for LLM reasoning on long videos. In *CVPR*, 2025c.
- Tianbao Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*, 2024.
- John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.
- Ce Zhang et al. LLoVi: A simple LLM framework for long-range video question-answering. In *EMNLP*, 2024.
- Luo Zhang et al. VideoGameBench: Can LMMs play video games? *arXiv preprint*, 2025.

A Full Per-Category Results

Table 9: Complete per-category results for all 8 observation configurations tested with Claude Sonnet 4.6.

Mode	A	B	C	D	E	F	G	H	I	J	Total
Standard	0	2	4	4	8	4	1	7	3	5	38
Uniform 1 FPS	0	4	10	8	9	8	1	8	6	4	58
Uniform 3 FPS	2	5	8	8	8	8	1	7	6	3	56
Pixel-diff	1	5	9	7	9	8	1	8	6	4	58
Random keyframes	1	6	8	6	9	8	1	8	6	3	56
AOI visual	1	5	9	7	8	9	1	8	6	4	58
AOI visual+ASR	1	4	10	9	9	9	4	9	6	3	64
AOI full	9	10	9	9	8	9	9	9	8	2	82

B DynaCU-Bench Task List

Each of the 100 tasks is identified by a category letter (A–J) and a difficulty-indexed ID. Easy tasks: E1, E2, E3. Medium tasks: M1, M2, M3, M4. Hard tasks: H1, H2, H3.

Example tasks by category:

- **A-E1** (Podcast, Easy): Listen to a podcast segment and identify the guest’s name.
- **B-M2** (Meeting, Medium): During a meeting with slides, count the number of items in a presented chart.
- **C-H1** (Video, Hard): Watch a terminal recording and describe the full workflow demonstrated.
- **D-M4** (Carousel, Medium): Identify a specific product name from a rotating carousel.
- **E-E3** (Dashboard, Easy): Read the current warning level from a live-updating dashboard.
- **F-E2** (Transient UI, Easy): Click “Accept” on a cookie consent banner before it auto-dismisses.
- **G-M1** (Phone, Medium): Follow spoken instructions during a simulated phone call.
- **H-H2** (Interview, Hard): Complete a multi-question spoken interview with scoring.
- **I-M2** (Collab, Medium): Resolve a conflict in a collaborative document when a remote edit arrives.
- **J-E2** (Games, Easy): Complete 3 rounds of a simple reaction-time game.

All tasks are implemented as self-contained HTML files with embedded JavaScript for task logic, audio synthesis, and evaluation. The complete benchmark (100 HTML files, evaluation scripts, and task definitions) will be released publicly.

C CLIP Threshold Calibration

The default threshold $\theta = 0.04$ was calibrated empirically. Web UI events such as modal dialogs appearing produce cosine distances of ~ 0.08 ; video scene cuts produce distances of ~ 0.15 – 0.25 ; loading spinners and cursor blinks produce distances of ~ 0.005 – 0.02 . Setting $\theta = 0.04$ captures all meaningful UI changes while filtering most periodic noise.

As shown in Figure 7, the method is robust across a wide range: any $\theta \in [0.02, 0.30]$ produces success rates within 80–90%. This insensitivity arises because CLIP embeddings produce a bimodal distribution of distances: semantically unchanged frames cluster near zero, while meaningful changes produce large distances, with relatively few samples in between.

D Implementation Details

Software. Python 3.11, Playwright 1.49 for browser automation, PulseAudio 16.1 for audio I/O, edge-TTS for high-quality speech synthesis, OpenAI CLIP ViT-B/16 for keyframe extraction, Whisper large-v3 for speech transcription, vLLM 0.19.0 for local model serving.

Hardware. All experiments run on a single machine with an NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM), 192 GB RAM. Cloud models (Claude, GPT-5.4, Gemini 2.5 Flash) are accessed via HTTP APIs. Local models (EvoCUA-32B, Fara-7B) are served via vLLM with 4-bit quantization (`bitsandbytes`). Whisper runs on CPU to avoid GPU contention.

Hyperparameters. Screen capture rate: ~ 3 Hz. Pixel change threshold (α): 1%. CLIP distance threshold (θ): 0.04. Maximum keyframes per step: 5. Audio buffer: 70 s ring buffer at 16 kHz mono. Post-action wait: 2.0 s. Maximum steps per task: 15.